

Reflexión técnica de la práctica CI/CD

1. Justificación de la estrategia Canary

Se seleccionó Canary para inventario-app porque una actualización de la API de inventario puede afectar operaciones de consulta y registro de productos. Antes de exponer una versión candidata a todo el tráfico, se verifica su comportamiento con una fracción acotada de solicitudes. La solución usa solo recursos nativos de Kubernetes: dos Deployments y un Service común. El Deployment estable mantiene cuatro réplicas y el canary una; ambos comparten la etiqueta app: inventario-app.

La proporción esperada es aproximadamente 80 % estable y 20 % canary. En 100 solicitudes se observaron 81 respuestas estables y 19 canary, resultado coherente con la relación 4:1. No es una cuota exacta por petición, pues el Service balancea conexiones. Frente a Blue-Green, Canary requiere una réplica adicional para la candidata y permite revertirla escalando ese Deployment a cero sin interrumpir la versión estable.

2. Métricas DORA verificables

Métrica	Datos y cálculo	Resultado	Nivel DORA
Lead time for changes	Cambio 1: 95 h 07 min 07 s. Cambio 2: 61 h 34 min 48 s. Promedio de ambos lead times.	78 h 20 min 58 s	Nivel alto: entre un día y una semana.
Frecuencia de despliegue	2 promociones exitosas en Minikube durante 1 día de medición.	2 despliegues por día	Cadencia diaria: nivel alto.
Change failure rate	0 promociones con rollback o corrección / 2×100 .	0 %	Nivel alto/élite: menor al 15 %.

3. Observación sobre persistencia de datos

Se creó un producto desde la interfaz y luego se eliminó el Pod con `kubectl delete pod`. Cuando Kubernetes creó el reemplazo, el producto dejó de aparecer. La aplicación guarda los datos en `data/products.json`, dentro del sistema de archivos del contenedor. El Pod nuevo inicia desde la imagen original y no dispone de un volumen persistente.

La pérdida observada no es un error que deba corregirse para esta práctica: evidencia el comportamiento esperado de una base de datos local en un contenedor efímero. En una

implementación productiva se requeriría una base de datos externa o almacenamiento persistente diseñado para escrituras concurrentes.

4. Bitácora de problemas reales encontrados y resueltos

Problema	Causa raíz	Solución aplicada
Trivy detectó una vulnerabilidad CRITICAL en el runtime.	La imagen mantenía npm y npx, herramientas innecesarias para iniciar el servidor y que ampliaban la superficie de paquetes.	Las pruebas permanecen en build-test; en runtime se eliminaron npm y npx. Trivy se ejecuta antes de publicar y falla ante CRITICAL.
El Pod no estaba disponible durante el arranque simulado.	STARTUP_DELAY_SECONDS=10 hace que /health responda 503 antes de que la aplicación esté lista.	Se incorporó startupProbe y se calibraron readinessProbe y livenessProbe. El Service espera la respuesta correcta.
La evidencia Canary no coincidía con los manifiestos entregados.	Las capturas se obtuvieron con nombres de manifiesto y configuración anteriores.	Se normalizaron los manifiestos, el candidato quedó fijado al SHA publicado y la evidencia documenta el reparto 4:1.

5. Conclusión

La práctica validó un flujo reproducible desde pruebas automatizadas hasta dos promociones controladas en Minikube. El lead time promedio fue 78 h 20 min 58 s, la frecuencia fue 2 despliegues por día y el change failure rate fue 0 %. Canary, las probes y Trivy aportan contención técnica, mientras la bitácora por SHA garantiza la trazabilidad de las métricas.